

# Recreating the FARM Scene Graph Pipeline on Spot

A Field Runbook: Nav-Graph Recording, Rosbag Capture, and Offline Scene-Graph Construction

Compiled from project notes (“Steps of recreation”)

September 15, 2026

## Abstract

This document is a step-by-step operational blueprint for reproducing the FARM-style 3D scene-graph mapping pipeline (see *FARM: How the scene graph is built from RGB-D + pose + camera info*, this project’s companion note) on a Boston Dynamics Spot robot. It consolidates the raw working notes captured during the original recreation effort into an ordered runbook: establishing the fiducial-anchored navigation graph, recording synchronized RGB-D + odometry data, streaming a live teleoperation bridge, and finally building and visualizing the offline scene graph from the captured frames. Known gaps and open engineering questions from the original notes are preserved verbatim in Section 5 so they are not lost in the reorganization.

## Contents

|          |                                                                     |          |
|----------|---------------------------------------------------------------------|----------|
| <b>1</b> | <b>Overview</b>                                                     | <b>2</b> |
| <b>2</b> | <b>Frame Conventions and the Fiducial Anchor</b>                    | <b>2</b> |
| <b>3</b> | <b>Prerequisites</b>                                                | <b>3</b> |
| <b>4</b> | <b>Step-by-Step Procedure</b>                                       | <b>3</b> |
| 4.1      | Step 1 — Record the Navigation Graph . . . . .                      | 3        |
| 4.2      | Step 2 — Upload the Navigation Graph . . . . .                      | 4        |
| 4.3      | Step 3 — Verify Waypoint-0 Localization in the Seed Frame . . . . . | 4        |
| 4.4      | Step 4 — Bring Up the ROS 2 / Docker Environment . . . . .          | 5        |
| 4.5      | Step 5 — Record the RGB-D + Odometry Rosbag . . . . .               | 5        |
| 4.6      | Step 6 (Optional) — Live Streaming Bridge + Viser Teleop . . . . .  | 6        |
| 4.7      | Step 7 — Extract Frames from the Rosbag . . . . .                   | 6        |
| 4.8      | Step 8 — Build the Scene Graph (Offline) . . . . .                  | 6        |
| 4.9      | Step 9 — Visualize the Scene Graph . . . . .                        | 7        |
| <b>5</b> | <b>Known Issues / Open Items</b>                                    | <b>7</b> |
| <b>6</b> | <b>Quick Command Reference</b>                                      | <b>8</b> |
|          | <b>Source</b>                                                       | <b>8</b> |

## 1 Overview

The goal of this pipeline is to produce a persistent, queryable 3D object-level scene graph from a Spot robot’s onboard sensors, in the *same spatial frame* as Spot’s own navigation graph (“nav graph”), so that scene-graph objects and navigation waypoints are directly comparable and composable downstream (e.g. for language-conditioned object-seeking navigation).

At a high level, three independent Spot/robotics subsystems have to be bridged:

1. **Spot’s GraphNav service** — records a fiducial-anchored navigation graph and exposes a **seed** frame that all waypoints are expressed in.
2. **ROS 2 sensor capture** — an aligned RGB-D + odometry stream, recorded to a rosbag (or streamed live) via a RealSense camera and Spot’s own odometry.
3. **The FARM-style offline scene-graph builder** — consumes extracted (**rgb**, **depth**, **intrinsics**, **pose**) frames and produces a persisted **scene\_state.pt** object map (see the companion pipeline note for the full 13-stage internals: segmentation, 3D Gaussian fitting, data association, VLM captioning, embeddings, etc.).

The critical integration point is **frame alignment**: odometry is captured in Spot’s local **odom** frame, but GraphNav waypoints (and therefore anything that needs to be comparable to the nav graph) live in the fiducial-anchored **seed** frame. Section 2 explains this transform chain, since getting it wrong is the single most common source of a misaligned scene graph.

## 2 Frame Conventions and the Fiducial Anchor

### Read this before recording anything

A physical **fiducial marker** (AprilTag-style) is used to localize both the navigation graph and the robot’s body frame at power-on. Waypoint 0 of the recorded nav graph is defined as wherever Spot is standing when GraphNav recording starts (its local **odom** origin, (0,0)), and the transform from that waypoint into the fiducial-anchored **seed** frame is what lets the scene graph and the nav graph share a common coordinate system. **The physical size of the fiducial marker must be configured correctly** in GraphNav / the fiducial detector — an incorrect size silently produces a scaled/incorrect **seed** transform.

The relevant transform chain is, top to bottom:

```

scene-graph object (camera frame)
    ↓  $T_{\text{cam} \rightarrow \text{body}}$ 
body frame
    ↓  $T_{\text{body} \rightarrow \text{odom}}$ 
odom frame (waypoint 0 = (0,0))
    ↓  $T_{\text{odom} \rightarrow \text{seed}}$ 
seed frame (shared with nav graph)
  
```

The middle transform ( $T_{\text{body} \rightarrow \text{odom}}$ ) comes from the robot’s own odometry topic; the final transform ( $T_{\text{odom} \rightarrow \text{seed}}$ ) comes from GraphNav’s recorded **seed\_tform\_body** at waypoint 0 (see Section 4.3). Re-projecting every scene-graph object through this chain is what places the FARM object map into the same frame as the nav graph.

### Why this matters operationally

- Spot **must be on the same network** as the machine running the scene-graph pipeline before any of the ROS topics or the GraphNav service are reachable.
- The fiducial's physical dimensions must match the configured size exactly, or the **seed** frame will be systematically wrong.
- Because the fiducial anchors localization, Spot should be re-localized **in the vicinity of the fiducial** any time it is power-cycled (see Section 4.2).

## 3 Prerequisites

- Spot robot, powered on and reachable at its hostname/IP (example used throughout this guide: 192.168.50.3), on the same VLAN as the Orin/edge compute unit driving the pipeline.
- A RealSense depth camera (used here via `realsense2_camera`'s `rs_launch.py` with `align_depth` enabled).
- A correctly-sized fiducial marker placed in the environment.
- Docker with the NVIDIA runtime (`-runtime nvidia -gpus all`) and an image named `spot_lio_sam:dev` containing the ROS 2 workspace (`rover_launch`, etc.).
- A Python virtual environment with the Spot SDK installed (`~/spot_sdk`), used for the standalone `graph_nav_command_line` / `recording_command_line` examples that ship with `spot-sdk`.
- The scene-graph repository (FARM-style offline builder — `scene_graph.offline.run` module, `rgb_bag_frame.py`, `scripts/view_scene_state.py`, `ros_ws_bridge_2.py`, `spot_graphnav_goal.py`) and its dependencies (viser for visualization / teleop UI).

## 4 Step-by-Step Procedure

The steps below are ordered for a full end-to-end run: establish the nav graph first (Steps 4.1–4.2), then capture sensor data (Step 4.5 for offline, or Step 4.6 for a live/teleop session), then build and visualize the scene graph (Steps 4.7–4.9).

### 4.1 Step 1 — Record the Navigation Graph

Before recording, confirm Spot can be pinged and is on the VLAN exposed on the Orin. Power Spot on **at the location that should become waypoint 0** (i.e. the robot's odometry origin), then run GraphNav's recording tool from the Spot SDK which is run in the python virtual environment. examples:

```
source ~/spot_sdk/bin/activate
(spot_sdk) indro@orin:~/spot-sdk/python/examples/graph_nav_command_line$ \
python3 -m recording_command_line \
--download-filepath ~/mist_ws_ros2/scene_graph/data_walk/nav_graph_spot_2 \
192.168.50.3
```

Drive/walk Spot through the area to be mapped using the recording tool's interactive commands; this produces the downloaded graph directory referenced in later steps.

While recording, the CLI presents several menu options. Start by clearing any locally cached map, then begin recording. It's important to do this first, then create the starting position as the default waypoint (option 4) — this lets you later query the transform from the seed frame to the odom frame. Also close any available loops and optimize the anchors before finishing.

## 4.2 Step 2 — Upload the Navigation Graph

Upload the recorded graph back onto Spot so it can relocalize against it. This is required any time Spot is power-cycled; upload while Spot is physically near the fiducial so it can re-localize against it immediately:

```
~/spot-sdk/python/examples/graph_nav_command_line$ \
python3 -m graph_nav_command_line \
--upload-filepath \
~/mist_ws_ros2/scene_graph/data_walk/nav_graph_spot_4/downloaded_graph/ \
192.168.50.3
```

## 4.3 Step 3 — Verify Waypoint-0 Localization in the Seed Frame

After recording, confirm the localization of waypoint 0 (the point where Spot was powered on) — this is the transform used to place the scene graph in the seed frame described in Section 2. Example output:

```
# get the localisation of the waypoint that is 0,0 of odom where Spot was powered on
waypoint_id: "lento-macaw-UIXPzApBFdLgdRhKVX0FuQ=="
waypoint_tform_body {
  position {
    x: -0.020189967017836097
    y: -0.01419134137423822
    z: 0.00901193025016922
  }
  rotation {
    x: 0.00020041028428302677
    y: 0.0068203531810062763
    z: -0.0089073042606859359
    w: 0.99993704929311034
  }
}
timestamp {
  seconds: 1787956904
  nanos: 469585920
}
seed_tform_body {
  position {
    x: 2.2729322207071028
    y: 0.19774609072594404
    z: -0.53609861630672606
  }
  rotation {
    x: 0.0047556974067231462
    y: 0.003794053310715498
    z: 0.99953135411525507
    w: 0.030001010685907416
```

```
}
}
```

`seed_tform_body` at waypoint 0 is exactly the  $T_{\text{odom} \rightarrow \text{seed}}$  transform from Section 2: it is what lets scene-graph poses (expressed relative to the odom origin) be re-projected into the fiducial-anchored seed frame shared with the nav graph.

The farm project has been modified to use se3 matrix transform, so use the script to convert the quaternion based transformation into the matrix.

```
WT=$(python scripts/quat_to_se3.py --xyz X Y Z --quat QX QY QZ QW --values-only)

python scripts/rgb_bag_frame.py ... --world-transform $WT
python scripts/view_scene_state.py ... --world-transform $WT
```

## 4.4 Step 4 — Bring Up the ROS 2 / Docker Environment

Every on-robot recording session (bag recording or live streaming) starts from the same containerized ROS 2 environment:

```
docker run -it --rm --network=host \
  -v /dev:/dev \
  --privileged \
  --name scene_graph_nav_graph \
  --device-cgroup-rule="a *:rmw" \
  --volume=/tmp/.X11-unix:/tmp/.X11-unix -v ${XAUTH}:${XAUTH} \
  -e XAUTHORITY=${XAUTH} \
  --runtime nvidia --gpus all \
  -v ${PWD}:/workspace \
  -w=/workspace \
  -e LIBGL_ALWAYS_SOFTWARE="1" \
  -e DISPLAY=${DISPLAY} \
  spot_luo_sam:dev
```

Inside the container, bring up the Spot driver and the aligned RealSense stream:

```
ros2 launch rover_launch spot_run.launch.py
ros2 launch realsense2_camera rs_launch.py align_depth.enable:=true
```

### Note

This same three-command sequence (`docker run` → `spot_run.launch.py` → `rs_launch.py`) is the shared prerequisite for both the rosbag recording path (Step 4.5) and the live WebSocket bridge path (Step 4.6) below — run it once per session, then branch into whichever capture mode you need.

## 4.5 Step 5 — Record the RGB-D + Odometry Rosbag

With the environment from Step 4.4 running, record the synchronized sensor bag that will later be converted into scene-graph frames:

```
ros2 bag record -o rosbags/scene_graph_spot_2 \
  /odometry \
  /camera/camera/color/image_raw \
```

```
/camera/camera/depth/camera_info \
/camera/camera/aligned_depth_to_color/camera_info \
/camera/camera/aligned_depth_to_color/image_raw
```

Drive Spot through the space to be mapped while this is recording. This is the offline-friendly capture path (bag first, build scene graph later — Steps 4.7–4.8).

## 4.6 Step 6 (Optional) — Live Streaming Bridge + Viser Teleop

As an alternative (or complement) to bag recording, a WebSocket bridge can stream Spot’s video and odometry live and accept navigation commands sent through a **viser**-based UI, driving Spot along the previously-recorded nav graph. This requires the same docker/ROS bring-up as Step 4.4, then:

```
python ros_ws_bridge_2.py --host 0.0.0.0 --port 8765 \
  --odom-topic /odometry \
  --image-topic /camera/camera/color/image_raw/compressed \
  --image-qos best_effort --image-max-fps 10 --image-max-side 640

python3 spot_graphnav_goal.py \
  --robot-hostname 192.168.50.3 \
  --graph-path /workspace/scene_graph/data_walk/nav_graph_spot_4/downloaded_graph/ \
  --ws-url ws://127.0.0.1:8765
```

Note that `spot_graphnav_goal.py` is pointed at the same downloaded graph directory uploaded in Step 4.2, so the nav graph must already be recorded and uploaded before this step is meaningful.

## 4.7 Step 7 — Extract Frames from the Rosbag

**Copy the rosbag and nav graph from spot to the server running the scene graph.**

To run **farm project**, `./run-temp.sh` shell to start the docker Convert the recorded bag (from Step 4.5) into the per-frame (rgb, depth, intrinsics, pose) tuples the offline scene-graph builder. Also add the transform from the odom frame to the seed frame — this is needed to express the camera’s odometry in the seed frame, so that object positions can later be queried in the nav graph’s frame. expects:

```
python rgb_bag_frame.py \
  --bag-dir ../data/scene_graph_spot_6/ \
  --out-dir ../data/spot_scene_graph_frames_6 \
  --rgb-topic /camera/camera/color/image_raw \
  --depth-topic /camera/camera/aligned_depth_to_color/image_raw \
  --camera-info-topic /camera/camera/aligned_depth_to_color/camera_info \
  --odometry-topic /odometry
```

## 4.8 Step 8 — Build the Scene Graph (Offline)

Run the offline scene-graph builder over the extracted frames directory. This is the FARM-style pipeline described in the companion pipeline note: per-frame open-vocabulary segmentation, 3D Gaussian fitting per detection, data association against existing objects, Gaussian/feature fusion, covisibility graph updates, and (with `-caption`) asynchronous VLM captioning and multi-modal embedding.

```
./run-temp.sh vllm #to start the models for captioning
python -m scene_graph.offline.run \
  --source frames-json \
  --frames-json-dir data/spot_scene_graph_frames_6 \
  --stride 5 \
  --save-path data/scene_graph_mist_lab_6_caption.pt \
  --covisibility \
  --caption
```

This produces the persisted `scene_state.pt`-style object map used by the viewer and downstream retrieval.

## 4.9 Step 9 — Visualize the Scene Graph

```
python scripts/view_scene_state.py \
  --pt data/scene_graph_mist_lab_6_caption.pt \
  --walk nav_graph_spot_4/ \
  --grid-cell-m 1.0 \
  --no-camera-trajectory \
  --ws-url ws://192.168.1.192:8765
```

Passing `-walk` alongside the saved scene-graph state overlays the recorded nav graph in the same viewer, which is the practical check that the frame-alignment work in Section 2 actually worked: objects and nav-graph waypoints should appear spatially consistent with each other.

The websocket url works only when spot and scene graph are on the same server with the client running on spot, to exchange information through the websocket,

### Known limitation at time of writing

Room/x-y boundary limits are not yet solvable from the pipeline itself — walls cannot currently be defined automatically (see Section 5). The `-grid-cell-m` flag only controls visualization grid resolution, it does not constrain or derive room extent.

## 5 Known Issues / Open Items

The following are open engineering questions and known gaps carried over verbatim from the working notes. They are not yet resolved and should be treated as the active punch list for anyone continuing this recreation effort.

- **Odometry in the vision frame:** investigate whether odometry can be obtained directly in the vision frame, which would remove the `cam_t_body` error currently present in the pipeline.
- **Camera recalibration:** the camera used to create the rosbag needs to be recalibrated.
- **Online version:** attempt an online (streaming, non-bag) version of the full pipeline rather than the current record-then-process offline flow.
- **Walls and boundaries:** figure out how to obtain/derive room walls and boundaries for the scene graph — not currently solved; the x/y room limits cannot yet be defined automatically (see the note under Step 4.9).
- **Navigable-area logic:** improve the navigable-area logic by tuning the relevant flags (specific flags to be identified/tuned empirically).

## 6 Quick Command Reference

| Stage                   | Command / Script                                                                                                                                          |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bring up ROS/Docker     | <pre>docker run ...spot_lio_sam:dev ros2 launch rover_launch spot_run.launch.py ros2 launch realsense2_camera rs_launch.py align_depth.enable:=true</pre> |
| Record nav graph        | <pre>python3 -m recording_command_line -download-filepath ...</pre>                                                                                       |
| Upload nav graph        | <pre>python3 -m graph_nav_command_line -upload-filepath ...</pre>                                                                                         |
| Record rosbag           | <pre>ros2 bag record -o rosbags/.../odometry ...</pre>                                                                                                    |
| Live bridge (optional)  | <pre>python ros_ws_bridge_2.py -host 0.0.0.0 -port 8765 ... python3 spot_graphnav_goal.py -robot-hostname ...-graph-path ...</pre>                        |
| Extract frames from bag | <pre>python rgb_bag_frame.py -bag-dir ...-out-dir ...</pre>                                                                                               |
| Build scene graph       | <pre>python -m scene_graph.offline.run -source frames-json ...-covisibility -caption</pre>                                                                |
| Visualize               | <pre>python scripts/view_scene_state.py -pt ...-walk ...-ws-url ...</pre>                                                                                 |

## Source

This runbook reorganizes and annotates the raw working notes from the project Notion page “*Steps of recreation*”. Commands and parameter values are reproduced as recorded; environment-specific values (IP addresses, file paths, waypoint IDs) should be substituted with the values for the specific Spot unit and workstation in use. For background on what the offline scene-graph builder (Step 4.8) does internally, see the companion project note “*FARM: How the scene graph is built from RGB-D + pose + camera info.*”